Learn Online, Ask Early, Match Globally: A Staged Policy for Sequential Reciprocal Introductions

Team: Laplace · **Author:** Kumar Ankit · **Round 1 research note** · Sequential Matching Problem

All data and outcomes referred to here are synthetic and come from the public development simulator. This note proposes a method. The only results reported are the baseline runs in §2, which I ran myself. No result is claimed for the proposed policy.

1. Problem interpretation

Every simulated day, the policy first spends up to 12 clarification units, then proposes a set of introductions. Each proposed pair must be reciprocally feasible, must not overlap with any other pair, and must not repeat an earlier introduction. The score is the mean, over six scenario families, of **Mutual Second-Meeting Intention (MSMI) per 100 arrived members**. A pair counts only if both people accept, a date happens within 30 days of assignment, and both people say Yes to a second meeting within 3 days of that date (PS §8, §10). The denominator is fixed, so waiting or leaving people unserved never improves the ratio. A single invalid action in any of the 120 private episodes makes the entry unranked (PS §7).

I read the task as three coupled decisions, each with a different kind of uncertainty:

1. **Who becomes matchable (clarification).** A pair with a missing hard field is blocked. A *declined* hard field blocks a member permanently. At most four constraint bundles fit into a day.
2. **Which feasible pairs to introduce today (allocation).** One person per batch. Each introduction ties both people up for days (8 in the public simulator), and each pair can be used only once. Choosing pairs one at a time can lose value compared with choosing them jointly (PS §2).
3. **Whether to wait.** Waiting can keep a person free for a better partner who has not yet arrived or is not yet clarified. Waiting also spends part of a 60-day horizon.

The outcome is rare and arrives late. Learning must therefore use earlier, higher-count proxies and must treat unresolved introductions as *censored*, not as failures (PS §8–9).

2. Evidence so far: baseline runs

Command: `python3 evaluate.py --baseline {greedy,no_asks,random} --seeds 101,102,103 --variants all`. This uses trusted local mode, 18 episodes per baseline, and all 54 episodes were valid. The table gives means over episodes. CIs come from a bootstrap that resamples seeds within each variant (`scripts/baseline_stats.py`).

Baseline	Primary MSMI/100 (95% CI)	MSMI events	Coverage	Mutual acc./100	Ask cost
greedy	0.389 [0.31, 0.50]	14	0.37	5.50	207
random-feasible	0.278 [0.14, 0.44]	10	0.38	5.33	207
no-clarification	0.139 [0.06, 0.22]	5	0.14	1.75	0

What these runs show:

- **Greedy vs random.** Greedy's point estimate is 0.11 MSMI/100 higher than random, about 40% relative. The paired 95% interval is [–0.03, 0.25] and includes zero, and the two per-policy intervals overlap. The effect may be real, but 3 seeds per variant cannot resolve it, and a bootstrap over 3 seeds is itself crude.
- **Clarification matters.** No-clarification's interval does not overlap greedy's, and its coverage is about 3× lower. It scores zero in both sparse and cold-start.
- **The funnel narrows very fast.** For greedy on the development variant, about 100 assignments lead to 13 mutual acceptances, then 10 dates, then 1 MSMI. That is roughly 1% of assignments. About 60 of about 200 directional responses are missing.
- **Coverage is capped near 43%.** About 113 of about 200 members are never introduced. In public seed 101, 53 members have at least one *declined* hard field and therefore can never become feasible.
- **The budget is not what limits clarification; the order is.** greedy and random spend about 200 of the 720 available units, because the baseline ask rule runs out of eligible members. Its first-in-list ordering only decides *how early* people become matchable. The unused units could pay for soft-field questions.
- **Sparse supply is the hardest case.** In the sparse variant all baselines reach only about 14% coverage and about 18 assignments per episode.

3. Hypotheses

- **H1 (clarification order).** Rank constraint bundles by the expected number of feasible partners they would unlock, weighted by days remaining, and skip members with a declined hard field. This should raise coverage and mutual acceptances over the baseline order, most of all in cold-start and sparse.
- **H2 (pair value).** Use directional acceptance estimates that treat soft-field agreement as evidence and treat "unknown" as its own state. This should raise mutual acceptances per assignment over equal-field counting.
- **H3 (global allocation).** Maximum-weight matching on the same edge values should beat greedy-by-rank allocation. The effect should be small in thick pools and larger where people compete for the same candidates.
- **H4 (online learning).** Weights updated within the episode, starting from weak priors, should be more robust than fixed priors in the *shift* and *drift* families, at little cost elsewhere.
- **H5 (waiting).** A waiting threshold that shrinks with the remaining horizon should help only modestly. Dynamic matching theory finds greedy timing close to optimal when departures cannot be anticipated [1, 2]. I expect H5 may be refuted and will report it either way.

4. Design principles taken from the programme workshops

Three programme workshops shaped the design:

- **W1:** *Exploration, Exploitation and Online Learning* (J. Liu, University of Edinburgh, 6 Oct 2026).
- **W2:** *Building Robust ML Systems from Messy Data*, a car-park demand forecasting case study.
- **W3:** *Hard Rules, Soft Goals*, a lecture with a student handout.

The table below lists the points I adopted and where each one appears in the plan.

Workshop point	Design decision in this proposal
W3: keep specification, prediction and decision separate. A post-solve validator checks feasibility before a decision is used. A learned score may feed the allocator but never overrides eligibility or capacity.	Separate feasibility layer, value model and allocator. Every batch is re-validated before output (§5, §6).
W3: write down which rules are hard and which goals are soft. A hard rule makes a plan impossible; a soft goal makes it worse.	**Hard:** reciprocal eligibility, availability, one person per batch, no repeat pair, daily ask budget. **Soft:** predicted outcome value, coverage, waiting time, ask cost. Soft terms never relax a hard rule.
W3: a *known forbidden* link differs from a link that has *not been observed*.	Three separate states: *infeasible* (excluded), *needs clarification* (an ask target), and *declined* (permanently blocked, never asked, never inferred).
W3: the multiplier λ on a binding budget is its shadow price.	The waiting threshold λ_a is read as the shadow price of person a's next 8-day slot. Stage c also tests λ taken from the dual of the fractional-matching LP.
W3: multiplying the two marginal acceptance probabilities needs conditional independence, and learning only from accepted matches biases estimates.	Joint-vs-product calibration check. Directional labels come from all responses, not just accepted pairs (§5).
W1: under *selective feedback* [14] a greedy policy can lock onto a wrong belief that looks well calibrated on its own data.	Thompson sampling over the posterior, restricted to feasible edges. The report states whether the learned weights actually moved.
W1: value of information is zero unless some possible answer changes the action [15]. Ask where uncertainty affects the decision, not where it is largest.	Soft-field asks are ranked by expected change in the chosen matching. A constraint bundle has zero value if no partner could become feasible.
W1: under non-stationarity, discount old evidence [16]. Newcomers need some introductions spent on learning.	A discount factor γ in the online updates (aimed at *drift*). The report breaks out waits and coverage for newcomers versus earlier arrivals.
W1: check that uncertainty estimates are calibrated.	Reliability tables for directional and joint acceptance on held-out rollouts.
W2: compare against baselines *and* a perfect-information ceiling. Validate on grouped hold-outs. Classify failures by type.	Analysis-only full-information reference (§9). Splits by world and seed, never by rows. A failure taxonomy for unserved members and funnel exits (§10).

W1 also covers matching-specific bandit work [17, 18]. Those papers study stable matching with two-sided learning rather than this platform-allocation objective, so I use them as background only.

The W1 peer question bank is participant-compiled and was not checked against the recording. It suggests off-policy (inverse-propensity) evaluation. I do **not** adopt this for the supplied files, because their propensities are null and PS §9 forbids claiming unbiased IPS from them. Policies are compared only in controlled simulator episodes.

I could not transcribe the W2 video, so only its slides were used.

5. Modelling approach

The code keeps **normalisation**, **modelling** and **allocation** separate (PS §15). Only the request JSON and policy-owned memory are inputs.

Feasibility layer. This is a pure function over observed fields that reproduces the published reciprocal rules (`kit.eligibility`), checks availability, and excludes previously introduced pairs. Every outgoing batch goes through a final validator for feasibility, distinct members, unused pairs and ask budget. If anything fails, the policy falls back to an empty, valid action.

Pair value. For each feasible edge (a, b), I estimate:

- **ρ_a: response propensity.** A Beta posterior per member. A deadline that passes with no response counts as "no recorded response". It is never treated as a No (PS §8).
- **p_ab: directional acceptance.** $p_{ab} = \sigma(\theta_0 + \Sigma_k w_k \cdot g_k(a,b) + u_a)$. Each g_k is a soft-field indicator with three states: agree, disagree, or unknown. u_a is a member-level effect fitted only from a's own observed Yes/No answers.
- **q_ab: post-acceptance factor.** P(date within 30 days, then two timely second-meeting Yes answers). It depends mainly on relationship-goal agreement and is fitted from the few dates observed.

The edge value is $V_{ab} \approx \rho_a \rho_b p_{ab} p_{ba} q_{ab}$. Multiplying the two directions assumes they are independent given the features. A shared pair-level component would break that (PS §3). I will check this by comparing the predicted joint acceptance against the product of the directional estimates on held-out rollouts. Probabilities are diagnostics only and do not affect ranking.

Training data and the rules that allow it. The policy treats simulator-trained weights as **weak priors**. **Online learning within each episode is the primary mechanism.** The relevant rules are:

- "*The simulator supports repeated episodes from day zero, so teams can collect their own decision histories under different policies.*" (PS §4)
- "*For model training, retain pre-action observations from simulator rollouts rather than attaching final profiles to earlier outcomes.*" (PS §9)
- "*Only observable state, clarification results, feedback, policy-owned memory and declared training assets are permitted inputs.*" and "*Training may use other compute; document it and include all assets required for offline inference.*" (PS §11)
- The limit that applies: "*Do not read hidden simulator objects, reconstruct hidden answers from public generator seeds or IDs … The inspectable public simulator is an experimental tool, not an oracle for the policy.*" (PS §11). Also the FAQ: inspecting the simulator source is allowed "*for reproducibility and experiments. Reconstructing hidden state or random outcomes inside a policy is prohibited.*"

Priors (μ_w, Σ_w) are fitted only from observations available before each action in my own rollouts on declared training seeds (§9). They ship as a small JSON asset. Generator coefficients are never copied in. Priors are deliberately broad, worth roughly 20 pseudo-observations per weight, so in-episode feedback can move or even flip them. Private worlds may come from a different engine (POLICY_INTERFACE.md allows "a compatible engine"), so priors are a starting point and not an assumption.

Online update. After each day's feedback, the policy updates the shared weights with a Laplace (diagonal-Gaussian) step on matured Yes/No responses. It also updates the member effects u_a, the Beta response posteriors, and a pooled estimate of q. The state stays compact (per-member counts plus a weight vector), well under the 1 MiB memory limit.

Learning targets, proxies and censoring. MSMI gives about one event per episode, too few to learn from directly. The model learns from earlier events with higher counts:

1. **Directional response:** a recorded Yes or No by the 7-day deadline.
2. **Mutual acceptance.**
3. **Date** within the window.
4. **Timely second-meeting Yes**, for q only.

An introduction whose observation window has not closed is **censored**: it is left out of the likelihood rather than counted as a negative. A missing response updates only ρ and never the acceptance model. A pair that never dated contributes no second-meeting label.

6. Decision and allocation strategy

Allocation routine (own code, no external dependencies). The starter kit is stdlib-only ("The starter has no external Python dependencies", README). PS §11 does permit added dependencies if they are pinned with licences. I still plan to ship a small matching routine of my own:

1. Greedy maximum-weight matching (a ½-approximation).

2. Local improvement until no gain remains: 2-edge swaps, i.e. replacing (a,b),(c,d) with (a,c),(b,d) or (a,d),(b,c), plus single-edge augmentations through free vertices.

The relevant ideas are Edmonds' blossom algorithm for exact matching [3, 4] and short augmentations in linear-time approximations [5]. Graphs here have at most ~200 nodes and a few hundred feasible edges, so this fits easily in the 10-second limit.

An **exact blossom solver is used only as a reference** in an offline test harness, never in the image. It measures the optimality gap of the shipped routine on logged daily graphs. If the gap is material, I will port an exact implementation into the policy as my own stdlib code.

Waiting rule. An edge enters the matching only if $V_{ab} - \lambda_a - \lambda_b > 0$. λ_a estimates a's opportunity cost: the value of a's best *other* currently feasible or soon-clarifiable partner, discounted by the 8-day lock-up. λ is set to zero when:

- fewer than about 10 decision days remain;
- a has at most one feasible alternative (thin market);
- a has waited longer than a cap, to protect coverage.

Coverage is the first tie-break, so the policy never leaves an available pair with positive value unmatched only to keep someone free.

7. Clarification policy

- **Constraint bundles (3 units).** Only for available members with no declined hard field. Members are ranked by *unlock value*: the expected number of currently feasible-if-clarified partners, using partners' observed constraints that apply to this member and this member's observed fields. That count is weighted by remaining days, with a bonus for scarce partners such as non-binary members or thin geographies.
- **Soft fields (1 unit) from the leftover budget.** Ask the field with the highest value of information: the expected change in the chosen matching if that field were known, estimated from the current weights. The prior ordering starts with relationship goal, but the learned weights decide.
- **Validity rules.** Never ask unavailable members. Never ask twice for an already observed or declined field. Never exceed `ask_budget_remaining`. Declined answers stay unknown and are never inferred.

8. Staged build plan with one ablation per stage (12–18 Oct)

The stages build on each other. Each one passes a gate on **validation seeds** before the next starts: the paired difference in mutual acceptances and dates per 100 must not be negative, and MSMI must not be worse beyond its CI.

- **Stage a: safe core (12–13 Oct).** Feasibility validator, crash-safe fallback, fast in-process evaluator, and the H1 clarification ranking, with greedy allocation kept as is. *Ablation A:* H1 ask order vs the baseline ask order, all else fixed.
- **Stage b: value model and global matching (14–15 Oct).** Directional pair value with weak priors, own matching routine, and soft-field asks. *Ablation B:* global matching vs greedy-by-rank on identical edge values (tests H3). Secondary: soft asks off.
- **Stage c: online learning, waiting and exploration (16–17 Oct).** Online Bayesian updates with a discount factor γ [16], Thompson sampling over the posterior [6, 7] restricted to feasible edges, and the waiting rule as a slot shadow price (W3). *Ablation C:* online learning off, with priors frozen (tests H4). Secondary: waiting off (H5); $\gamma = 1$ vs $\gamma < 1$, aimed at *drift*; posterior mean vs sampling; heuristic λ vs LP-dual λ.
- **Freeze (18 Oct).** Container build, the full evaluation set, report, and the immutable commit SHA.

Exploration only ever chooses among *feasible* pairs. Hard constraints are never explored.

9. Experiments, baselines and metrics

- **Seeds and splits.**
- Prior fitting: training rollout seeds 1000–1999.
- Stage gates: validation seeds 2000–2099.
- Reported results: held-out test seeds 101–120, with all six variants for every policy.
- Static day-30 pools: used only for exploratory analysis, keeping the manifest split (train 01–06, validation 07–08, test 09–10).
- The inference random seed is fixed and declared.
- **Comparisons.** Greedy, no-clarification, random-feasible and my policy on identical seeds and variants (PS §10). Primary statistics are paired per-seed differences with bootstrap CIs and Poisson-aware intervals on event counts.
- **Metrics, reported per variant.** MSMI/100 (primary); coverage; the full funnel (assignments, mutual acceptances, dates, MSMI); first-introduction waits and the unserved count; ask cost; missing feedback; inference time. Also

reported: waits and coverage for newcomers vs day-0 arrivals.

- **Diagnostics only.**
- Calibration of directional and joint acceptance estimates on held-out rollouts.
- Optimality gap of the matching routine against the exact solver.
- **Analysis-only full-information reference (W2).** This follows W2's "perfect-forecast ceiling". In a separate offline experiment, the harness reveals every *disclosed* hard field at arrival, and I run the same policy. The gap to the normal run bounds how much MSMI is lost to missing information, and so how much clarification can still gain. This harness change is never part of the submitted policy, which sees only the public request JSON.
- **Failure taxonomy (W2).**
- Each unserved member is classed as: blocked by a declined field; no feasible partner; feasible but never chosen; or departed before being served.
- Each introduction is classed by its funnel exit: no response, a No, no date, date after 30 days, or second-meeting answer late or No.
- **Validity.** `python -m unittest`, `verify_data.py`, and container-mode evaluation (`--image`) on every candidate, with zero invalid episodes required.

10. Expected failure modes and mitigations

Failure mode	Why it can happen	Mitigation / what I will report
Invalid action, crash, timeout or memory over 1 MiB	One invalid private episode makes the entry ineligible	Final validator; try/except fallback to an empty valid action; compact memory; container runs with a timing margin
Sparse supply	Few feasible edges; waiting starves people	λ set to 0 for thin-market members; report unserved members separately
Withheld (declined) constraints	Member can never be matched	Skip those asks; report blocked members as a separate unserved category
Competition for the same candidate	Popular members attract many top edges; the rest wait	Global matching; coverage-aware waiting cap; report the wait distribution
Delayed feedback and late dates	Dates can fall outside the 30-day window; labels censored	Censoring-aware likelihood; q learned per family; late-horizon behaviour analysed
Shift and drift	Priors can have the wrong sign or go stale	Weak priors plus online updates; Ablation C shows whether this works
Cold start	Few labels early	Priors and member effects start at 0; posterior uncertainty drives exploration
Too few labels for online learning	About 150 directional labels per episode	Shrinkage to priors; report when learned weights do not move
Wrong independence assumption	A shared pair component makes the product biased	Compare predicted joint vs product on held-out data
Overfitting to the public simulator	Private engine or worlds may differ	No generator constants; broad priors; robustness across all six variants
Confidently wrong model (W1)	Selective labels let a greedy learner lock in a wrong belief	Thompson sampling; calibration tables; report whether learned weights moved
Noisy conclusions	About 1 MSMI event per episode	Proxies, paired seeds, at least 20 seeds per variant, CIs on every claim

11. Scope, honesty and tools

Simulator scores measure only this invented environment. They say nothing about real relationships (PS §1, §14). The only numbers in this note are the baseline runs in §2.

Design principles from the programme's three workshops (W1–W3, §4) are credited where used. AI coding assistance (Anthropic's Claude, via Claude Code) helped me read the documentation, run the baseline commands, and draft and edit this note. I reviewed all content. Every reference below was checked against its publisher or indexing record.

References

1. M. Akbarpour, S. Li, S. Oveis Gharan. Thickness and Information in Dynamic Matching Markets. *Journal of Political Economy* 128(3):783–815, 2020.
2. R. Anderson, I. Ashlagi, D. Gamarnik, Y. Kanoria. Efficient Dynamic Barter Exchange. *Operations Research* 65(6):1446–1459, 2017.

3. J. Edmonds. Paths, Trees, and Flowers. *Canadian Journal of Mathematics* 17:449–467, 1965.
4. J. Edmonds. Maximum Matching and a Polyhedron with 0,1-Vertices. *Journal of Research of the National Bureau of Standards* 69B:125–130, 1965.
5. D. E. Drake, S. Hougardy. A Simple Approximation Algorithm for the Weighted Matching Problem. *Information Processing Letters* 85:211–213, 2003.
6. O. Chapelle, L. Li. An Empirical Evaluation of Thompson Sampling. *NIPS 24*, 2011.
7. D. Russo, B. Van Roy, A. Kazerouni, I. Osband, Z. Wen. A Tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning* 11(1), 2018.
8. P. Joulani, A. György, C. Szepesvári. Online Learning under Delayed Feedback. *ICML*, PMLR 28:1453–1461, 2013.
9. O. Chapelle. Modeling Delayed Feedback in Display Advertising. *KDD*, 2014.
10. I. Ashlagi, A. K. Krishnaswamy, R. Makhijani, D. Saban, K. Shiragur. Assortment Planning for Two-Sided Sequential Matching Markets. *Operations Research* 70(5):2784–2803, 2022.
11. I. Ashlagi, M. Braverman, Y. Kanoria, P. Shi. Clearing Matching Markets Efficiently: Informative Signals and Match Recommendations. *Management Science* 66(5):2163–2193, 2020.
12. L. Pizzato, T. Rej, T. Chung, I. Koprinska, J. Kay. RECON: A Reciprocal Recommender for Online Dating. *RecSys*, 207–214, 2010.
13. I. Ashlagi, M. Burq, C. Dutta, P. Jaillet, A. Saberi, C. Sholley. Edge-Weighted Online Windowed Matching. *Mathematics of Operations Research* 48(2):999–1016, 2023 (EC 2019).
14. H. Lakkaraju, J. Kleinberg, J. Leskovec, J. Ludwig, S. Mullainathan. The Selective Labels Problem: Evaluating Algorithmic Predictions in the Presence of Unobservables. *KDD*, 2017.
15. R. A. Howard. Information Value Theory. *IEEE Transactions on Systems Science and Cybernetics* 2(1):22–26, 1966.
16. A. Garivier, E. Moulines. On Upper-Confidence Bound Policies for Switching Bandit Problems. *ALT*, 2011.
17. S. Das, E. Kamenica. Two-Sided Bandits and the Dating Market. *IJCAI*, 947–952, 2005.
18. L. T. Liu, H. Mania, M. I. Jordan. Competing Bandits in Matching Markets. *AISTATS*, PMLR 108:1618–1628, 2020.

Programme materials (not peer-reviewed):

- W1: J. Liu, *Exploration, Exploitation and Online Learning*, slides, 6 Oct 2026. W1 peer question bank, participant-compiled, 6 Oct 2026.
- W2: *Building Robust ML Systems from Messy Data: Forecasting Car Park Demand*, slides.
- W3: *Hard Rules, Soft Goals: Physics, Ecosystems and Constrained Learning*, lecture slides and student handout.

How they are used:

- [8, 9]: handling delayed and censored labels.
- [10–12]: reciprocal, two-sided recommendation.
- [13]: batched maximum-weight matching when agents leave after a window, which motivates re-solving a global matching each day.